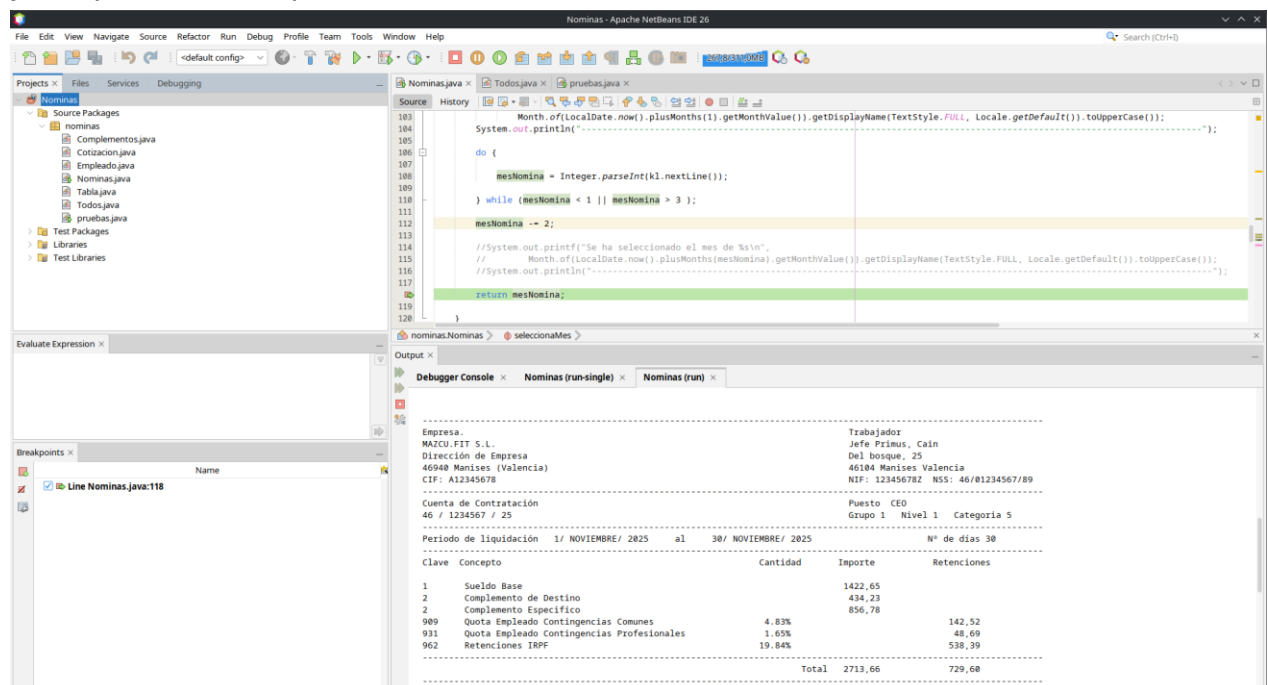


Desarrollo del Sistema de Gestión de Nóminas en Java (Apache NetBeans)

En la fase de desarrollo del ciclo de vida del software, se implementó un sistema de gestión de nóminas empresariales utilizando el lenguaje de programación Java y el entorno de desarrollo Apache NetBeans. Este sistema está diseñado como una aplicación modular y orientada a objetos, compuesta por múltiples clases que encapsulan responsabilidades específicas para mantener un código estructurado y mantenible. **Nominas.java** actúa como la clase principal, gestionando la interacción con el usuario mediante un menú de consola que permite seleccionar empleados, meses de liquidación y generar nóminas detalladas, mientras que clases como **Empleado**, **Tabla** y **Complementos** manejan los datos de empleados, tablas salariales y complementos respectivamente.



La arquitectura del sistema sigue principios de **cohesión y bajo acoplamiento**, donde cada clase tiene una responsabilidad única: **Todos** se encarga de la inicialización y almacenamiento de empleados, **Tabla** gestiona las escalas salariales del año 2025, y **Cotizacion** maneja los tramos y bases de cotización a la Seguridad Social. Esto permite que futuras modificaciones, como actualizaciones en las tablas salariales o cambios en los cálculos de cotizaciones, puedan realizarse de forma localizada sin afectar otras partes del sistema.

Durante el desarrollo, se hizo especial énfasis en la **validación de entrada de datos** y la **robustez del sistema**. Por ejemplo, en la selección de meses y empleados, se implementaron bucles de control que garantizan que el usuario introduzca valores válidos, evitando errores en tiempo de ejecución. Además, el sistema calcula

automáticamente valores como la base de cotización, las retenciones del IRPF y las cuotas a la Seguridad Social, asegurando precisión en los resultados financieros.

La implementación incluye funcionalidades avanzadas como el **manejo dinámico de fechas** mediante la API `java.time`, que permite generar nóminas para meses anteriores, actuales o futuros de forma flexible. La salida de las nóminas se presenta en un formato claro y estructurado en consola, detallando tanto los devengos como las deducciones, lo que facilita su revisión y validación. Finalmente, el uso de Apache NetBeans permitió una gestión eficiente del proyecto, con herramientas integradas para depuración, organización de paquetes y generación de documentación, culminando en un sistema funcional y listo para su despliegue en entornos productivos.

La clase `pruebas.java` actúa como un **entorno de pruebas aislado** durante el desarrollo del sistema de nóminas, permitiendo validar componentes individuales sin ejecutar toda la aplicación. Este enfoque sigue la metodología de **"prueba por unidades"** dentro del ciclo de desarrollo, donde cada método o funcionalidad se verifica de forma independiente antes de integrarlo en el sistema principal.

En este caso específico, `pruebas.java` contiene código para **seleccionar y validar meses** de nómina, lo que originalmente permitió al desarrollador:

1. **Probar la lógica de selección de fechas** sin cargar todo el sistema de nóminas
2. **Verificar el formato de salida** de los nombres de meses
3. **Depurar interacciones con el usuario** en un entorno controlado
4. **Validar cálculos temporales** con `LocalDate` antes de integrarlos en `Nominas.java`

Esta práctica es especialmente útil en **Apache NetBeans**, donde se puede ejecutar rápidamente cualquier clase con método `main` mediante clic derecho → "Run File" (Shift+F6), facilitando iteraciones rápidas de desarrollo. Una vez que la funcionalidad está depurada en `pruebas.java`, se migra al código principal, manteniendo la clase de pruebas como referencia o para futuras modificaciones.

Beneficios de este enfoque:

1. **Reducción de complejidad:** No es necesario navegar por menús completos para probar una función
2. **Depuración focalizada:** Los errores se aíslan más fácilmente
3. **Desarrollo incremental:** Se construye el sistema pieza por pieza con validación constante
4. **Documentación viva:** La clase de pruebas sirve como ejemplo de uso de los métodos desarrollados

Esta estrategia refleja una **buena práctica de programación**, especialmente en proyectos